COSC490LLMs

Homework 3: Building Your Neural Network!

Name: Dustin O'Brien
Collaborators, if any: None
Sources used for your homework, if any: Currently None

This assignment is focusing on understanding the fundamental properties of neural networks and their training.

Homework goals: After completing this homework, you should be comfortable with:

- thinking about neural networks
- key implementation details of NNs, particularly in PyTorch,
- explaining and deriving Backpropagation,
- debugging your neural network in case it faces any failures.

How to hand in your written work: via MyClasses.

Collaboration: Make certain that you understand the course collaboration policy, described on the course website. You may discuss the homework to understand the problems and the mathematics behind the various learning algorithms, but you are **not allowed to share problem solutions with any other students. You must write the solutions individually**.

Typesetting: We strongly recommend typesetting your homework, especially if you have sloppy handwriting. We will provide a LaTeX template for homework solutions. Type your answers into the corresponding solution field under each question.

1 Concepts, intuitions and big picture

1. Suppose you have built a neural network. You decide to initialize the weights and biases to be zero. Which of the following statements are True? (Check all that apply)
   ☒ Each neuron in the first hidden layer will perform the same computation. So even after multiple iterations of gradient descent each neuron will be computing the same thing as other neurons in the same layer.
   ☐ Each neuron in the first hidden layer will perform the same computation in the first iteration. But after one iteration of gradient descent they will learn to compute different things because we have "broken symmetry".
   ☐ Each neuron in the first hidden layer will compute the same thing, but neurons in different layers will compute different things, thus we have accomplished "symmetry breaking" as described in lecture.
   ☐ The first hidden layer's neurons will perform different computations from each other even in the first iteration; their parameters will thus keep evolving in their own way.
   Answer:

2. Vectorization allows you to compute forward propagation in an L-layer neural network without an explicit for-loop (or any other explicit iterative loop) over the layers $l = 1, 2, \times, L$. True/False?
   ☒ True
   ☐ False
   Answer:

3. The `tanh` activation usually works better than sigmoid activation function for hidden units because the mean of its output is closer to zero, and so it centers the data better for the next layer. True/False?
   ☒ True
   ☐ False
   Answer:

4. Which of the following techniques does NOT prevent a model from overfitting?
   ☐ Data augmentation ☐ Dropout ☐ Early stopping ☒ None of the above
   Answer:

5. Why should dropout be applied during training? Why should dropout NOT be applied during evaluation?
   Answer: *The number of parameters and scale of NN allows for massive degrees of overfitting. Dropout is used to prevents NN to be overreliant and "Memorizing" certain patterns in the data. Instead by dropping out certain aspects of network prevents overreliance and forces the "understanding" of general patterns. This Is only practical during the training phase as that is when these parameters are being chosen. If dropout were applied during evaluation it would create inconsistencies in the results of the data as well as not truly testing the model but simply a subset or similar model.*

6. Explain why initializing the parameters of a neural net with a constant is a bad idea.
   Answer: *Similar to the previously mentioned problem of setting all to 0 by putting all parameters to the same constant will result in the same operation being performed on all parameters and will make each layers parameters identical to another.*

7. You design a fully connected neural network architecture where all activations are sigmoids. You initialize the weights with large positive numbers. Is this a good idea? Explain your answer.
   Answer: *No this will create something similar to constant problem. Since the sigmoid function is capped between 0-1 large numbers will generally be mapped to 1 essentially setting it as a the constant 1.*

8. Explain what is the importance of "residual connections".
   Answer: *Residual connections is a way to reapply weight of information earlier inside of the neural network later on this decreases the negative impact that vanishing gradients have on the model. allowing for a more accurate impact on the model.*

9. What is cached ("memoized") in the implementation of forward propagation and backward propagation?
   ☒ Variables computed during forward propagation are cached and passed on to the corresponding backward propagation step to compute derivatives.
   ☐ Caching is used to keep track of the hyperparameters that we are searching over, to speed up computation.
   ☐ Caching is used to pass variables computed during backward propagation to the corresponding forward propagation step. It contains useful values for forward propagation to compute activations.
   Answer:

10. Which of the following statements is true?
☒ The deeper layers of a neural network are typically computing more complex features of the input than the earlier layers.
☐ The earlier layers of a neural network are typically computing more complex features of the input than the deeper layers.
Answer:

2 Revisiting Jacobians

Recall that Jacobians are generalizations of multi-variate derivatives and are extremely useful in denoting the gradient computations in computation graph and Backpropagation. A potentially confusing aspect of using Jacobains is their dimensions and so, here we're going focus on understanding Jacobian dimensions.

Recap: Let's first recap the formal definition of Jacobian. Suppose $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$ is a function takes a point $\mathbf{x} \in \mathbb{R}^n$ as input and produces the vector $\mathbf{f}(\mathbf{x}) \in \mathbb{R}^m$ as output. Then the Jacobian matrix of $\mathbf{f}$ is defined to be an $m \times n$ matrix, denoted by $\mathbf{J_f}(\mathbf{x})$, whose (i, j)th entry is $\mathbf{J}_{ij} = \frac{\partial f_i}{\partial x_j}$, or:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial \mathbf{f}}{\partial x_1} & \cdots & \frac{\partial \mathbf{f}}{\partial x_n} \end{bmatrix} = \begin{bmatrix} \nabla^\top f_1 \\ \vdots \\ \nabla^\top f_m \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

Examples: The shape of a Jacobian is an important notion to note. A Jacobian can be a vector, a matrix, or a tensor of arbitrary ranks. Consider the following special cases:

- If f is a scalar and $\mathbf{w}$ is a $d \times 1$ column vector, the Jacobian of f with respect to $\mathbf{w}$ is a row vector with $1 \times d$ dimensions.
- If $\mathbf{y}$ is a $n \times 1$ column vector and $\mathbf{z}$ is a $d \times 1$ column vector, the Jacobian of $\mathbf{z}$ with respect to y, or $\mathbf{J_z}(\mathbf{y})$ is a $d \times n$ matrix.
- Suppose $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{l \times p \times q}$. Then the Jacobian $\mathbf{J_A}(\mathbf{B})$ is a tensor of shape $(m \times n) \times (l \times p \times q)$. More broadly, the shape of the Jacobian is determined as (shape of the output)×(shape of the input).

Problem setup: Suppose we have:

- X, an $n \times d$ matrix, $x_i \in \mathbb{R}^{d \times 1}$ correspond to the rows of $X = [x_1, \ldots, x_n]^\top$
- Y, a $n \times k$ matrix
- W, a $k \times d$ matrix and w, a $d \times 1$ vector

For the following items, compute <u>(1) the shape of each Jacobian, and (2) an expression for each Jacobian</u>:

1. $f(w) = c$ (constant)
Answer:
$$\frac{d}{dw}(f(w)) = 0$$

2. $f(w) = \|w\|^2$ (squared L2-norm)
Answer:
$$\frac{d}{dw}(f(w)) = 2w$$

3. $f(w) = w^\top x_i$ (vector dot product)
   Answer:
$$\frac{d}{dw}\left(f(w)\right) = x_i$$

4. $f(w) = Xw$ (matrix-vector product)
   Answer:
$$\frac{d}{dw} = X$$

5. $f(w) = w$ (vector identity function)
   Answer:
$$\begin{bmatrix} 1 \\ 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}$$

6. $f(w) = w^2$ (element-wise power)
   Answer: $2w$

7. **Extra Credit:** $f(W) = XW^\top$ (matrix multiplication)
   Answer:
$$\frac{dX}{dW} = X$$

3 Activations Per Layer, Keeps Linearity Away!

Based on the content we saw at the class lectures, answer the following:

1. Why are activation functions used in neural networks? Answer: *Because not all functions are linearly separable and therefore require a different activatioon function to be properly modeled by a neural network. in other words it increases the expressive power of the neural network. :)*

2. Write down the formula for three common action functions (sigmoid, ReLU, Tanh) and their derivatives (assume scalar input/output). Plot these activation functions and their derivatives on $(-\infty, +\infty)$. Answer: *Sigmoid functions*

$$f(x) = \frac{1}{1 - e^{-x}}$$

$$f'(x) = \frac{e^{-x}}{(1 - e^{-x})^2}$$

ReLU Function

$$f(x) = \max(0, x)$$

$$f'(x) = \begin{cases} 1 & : x > 0 \\ 0 & : x < 0 \end{cases}$$

Tanh Function

$$f(x) = \tanh(x)$$

$$f'(x) = 1 - \tanh^2(x)$$

3. What is the "vanishing gradient" problem? (respond in no more than 3 sentences) Which activation functions are subject to this issue and why? (respond in no more than 3 sentences).
Answer: *Vanishing gradients is a problem where as you move backwards through a network the gradients tend to shrink and therefore slow down gradient descent and the program all together. It is overall a problem with functions that saturate including* tanh *and Sigmoid.*

4. Why zero-centered activation functions impact the results of Backprop?
Answer: *Zero centered functions are generallt problematic because it damages the ability for a function to move in opposite directions allowing for both sides allows for a wide range of mtions that a vector can have on its output*

5. Remember the Softmax function $\sigma(\mathbf{z})$ and how it extends sigmoid to multiple dimensions? Let's compute the derivative of Softmax for each dimension. Prove that:

$$\frac{d\sigma(\mathbf{z})_i}{dz_j} = \sigma(\mathbf{z})_i(\delta_{ij} - \sigma(\mathbf{z})_j)$$

where δ_{ij} is the Kronecker delta function.* Answer: *Firstly we will solve assuming i==j*

$$\frac{d}{dx}\sigma(\mathbf{z}) = \frac{d}{dx}\frac{e^{z_i}}{\sum_{k=1}^{n} e^{z_k}}$$

$$e^{z_i}\left(\sum_{k=1}^{n} e^{z_k}\right)^{-1} - e^{z_i}e^{z_i}\left(\sum_{k=1}^{n} e^{z_k}\right)^{-2}$$

$$e^{z_i}\left(\sum_{k=1}^{n} e^{z_k}\right)^{-1}\left(1 - e^{z_i}\left(\sum_{k=1}^{n} e^{z_k}\right)^{-1}\right)$$

$$\sigma_i(\mathbf{z})\left(1 - \sigma_j(\mathbf{z})\right)$$

Then secondly we assume i!=j

$$\frac{d}{dx}\sigma(\mathbf{z}) = \frac{d}{dx}\frac{e^{z_i}}{\sum_{k=1}^{n} e^{z_k}}$$

*https://en.wikipedia.org/wiki/Kronecker_delta

$$\frac{d}{dx} e^{z_i} \left(\sum_{k=1}^{n} \right)^{-1} - e^{z_i}$$

$$-e^{z_i} \left(\sum_{k=1}^{n} \right)^{-2} e^{z_j}$$

$$-\sigma_i(\mathbf{z})\,\sigma_j(\mathbf{z})$$

$$\sigma_i(\mathbf{z})\,(0 - \sigma_j \mathbf{z})$$

This shows that we fulfill the cases for the krunker delta and can insert it for both 1 and 0 in the respective functions

6. Use the above point to prove that the Jacobian of the Softmax function is the following:

$$\mathbf{J}_\sigma(\mathbf{z}) = \text{diag}(\sigma(\mathbf{z})) - \sigma(\mathbf{z})\sigma(\mathbf{z})^\top$$

where diag(.) turns a vector into a diagonal matrix. Also, note that $\mathbf{J}_\sigma(\mathbf{z}) \in \mathbb{R}^{K \times K}$..
Answer: *Firstly lets consider a diagonal where i == j*

$$\sigma_i(e^{\mathbf{z}_i}) - \sigma_i(e^{z_i})\,\sigma_j(e^{z_i})$$

$$\sigma_i(e^{\mathbf{z}_i})\,(1 - \sigma_j(e^{z_i}))$$

Which is our expected result

$$0 - \sigma_i(e^{z_i})\,\sigma_j(e^{z_i})$$

$$\sigma_i(e^{\mathbf{z}_i})\,(0 - \sigma_j(e^{z_i}))$$

Therefore the formula holds.

4 Simulating XOR

1. Can a single-layer network simulate (represent) an XOR function on $\mathbf{x} = [x_1, x_2]$?

$$y = \text{XOR}(\mathbf{x}) = \begin{cases} 1, & \text{if } \mathbf{x} = (0,1) \text{ or } \mathbf{x} = (1,0) \\ 0, & \text{if } \mathbf{x} = (1,1) \text{ or } \mathbf{x} = (0,0). \end{cases}$$

Explain your reasoning using the following single-layer network definition: $\hat{y} = \text{ReLU}(W \cdot \mathbf{x} + b)$ Answer: *First lets assume it does hold in which case we must have solutions to the following equations*

$$W_0 x_0 + w_1 x_1 + b = 0$$

$$W_0 x_0 + b = 1$$

$$W_1 x_1 + b = 1$$

$$b = 0$$

$$W_0 x_0 = 1 - b$$

$$W_1 x_1 = 1 - b$$

Plugging that into the first equation we generate

$$1 - b + 1 - b = 0$$

$$2 = 2b$$

$$b = 1$$

This a contradiction with $b = 0$

2. Repeat (1) with a two-layer network:

$$\mathbf{h} = \text{ReLU}(W_1 \cdot \mathbf{x} + \mathbf{b}_1)$$
$$\hat{y} = W_2 \cdot \mathbf{h} + b_2$$

Note that this model has an additional layer compared to the earlier question: an input layer $\mathbf{x} \in \mathbb{R}^2$, a hidden layer $\mathbf{h}$ with ReLU activation functions that are applied component-wise, and a linear output layer, resulting in scalar prediction $\hat{y}$. Provide a set of weights W_1 and W_2 and biases $\mathbf{b}_1$ and b_2 such that this model can accurately model the XOR problem.
Answer: *An answer for this would be*

$$W_1 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} b_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} W_2 = \begin{bmatrix} 1 & -1 \end{bmatrix} b_2 = 0$$

Doing the respective calculations we get the following

$$ReLU(W \cdot \begin{bmatrix} 0 \\ 0 \end{bmatrix} + b) = 0$$

$$ReLU(W \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix} + b) = 1$$

$$ReLU(W \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} + b) = 1$$

$$ReLU(W \cdot \begin{bmatrix} 1 \\ 1 \end{bmatrix} + b) = 0$$

3. Consider the same network as above (with ReLU activations for the hidden layer), with an arbitrary differentiable loss function $\ell : \{0, 1\}\ \{0, 1\} \to \mathbb{R}$ which takes as input $\hat{y}$ and y, our prediction and ground truth labels, respectively. Suppose all weights and biases are initialized to zero. Show that a model trained using standard gradient descent will not learn the XOR function given this initialization.
Answer: *Gradient Descent will fail here as setting everything to 0 is essentially the 0 function and every index will be 0 so all indexes will have following calculations*

$$f(x) = 0$$
$$f'(x) = 0$$

and wont be trained

5 Neural Nets and Backpropagation

Draw the computation graph for $f(x, y, z) = \ln x + \exp(y) \cdot z$. Each node in the graph should correspond to only one simple operation (addition, multiplication, exponentiation, etc.). Then we will follow the forward and backward propagation described in class to estimate the value of f and partial derivatives $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$ at $[x, y, z] = [1, 3, 2]$. For each step, show you work.

> A computation graph, so elegantly designed
> With nodes and edges, so easily combined
> It starts with inputs, a simple array
> And ends with outputs, in a computationally fair way
>
> Each node performs, an operation with care
> And passes its results, to those waiting to share
> The edges connect, each node with its peers
> And flow of information, they smoothly steer
>
> It's used to calculate, complex models so grand
> And trains neural networks, with ease at hand
> Backpropagation, it enables with grace
> Making deep learning, a beautiful race
>
> –ChatGPT Feb 3 2023

1. Draw the computation graph for $f(x, y, z) = \ln x + \exp(y) \cdot z$. The graph should have three input nodes for x, y, z and one output node f. Label each intermediate node h_i.

   Answer:

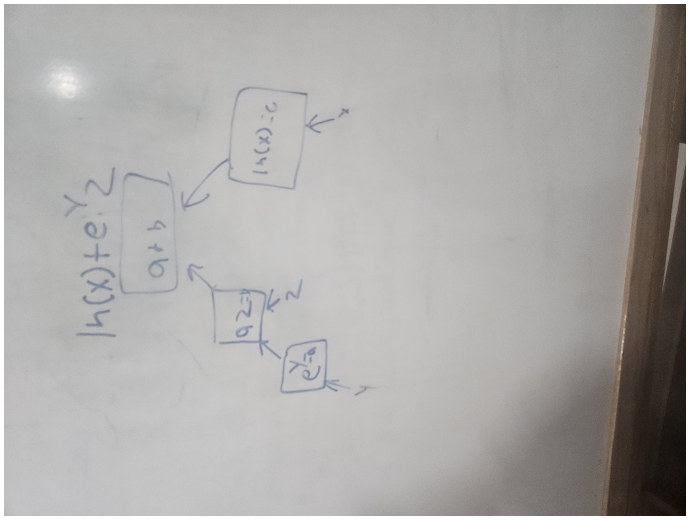


Figure 1: Computation graph for $f(x, y, z)$

2. Run the forward propagation and evaluate f and h_i $(i = 1, 2, \dots)$ at $[x, y, z] = [1, 3, 2]$.
   Answer:

$$h_1 = \ln x = \ln 1 = 0$$
$$h_2 = e^y = e^3$$
$$h_3 = h_2 \cdot z = e^3 \cdot 2 = 2e^3$$
$$f = h_1 + h_3 = 0 + 2e^3 = 2e^3$$

3. Run the backward propagation and give partial derivatives for each intermediate operation, i.e., $\frac{\partial h_i}{\partial x}$, $\frac{\partial h_j}{\partial h_i}$, and $\frac{\partial f}{\partial h_i}$. Evaluate the partial derivatives at $[x, y, z] = [1, 3, 2]$.
   Answer:

$$\frac{\partial h_1}{\partial x} = \frac{1}{x}$$
$$\frac{\partial h_2}{\partial y} = e^y$$
$$\frac{\partial h_3}{\partial h_2} = z$$
$$\frac{\partial h_3}{\partial z} = h_2$$

$$\frac{\partial f}{\partial h_1} = 1$$

4. Aggregate the results in (c) and evaluate the partial derivatives $\frac{\partial f}{\partial x}$, $\frac{\partial f}{\partial y}$, $\frac{\partial f}{\partial z}$ using the chain rule. Show your work.
   Answer:

$$\frac{\partial h_1}{\partial x}(1) = 1$$

$$\frac{\partial h_2}{\partial y}(3) = e^3$$

$$\frac{\partial h_3}{\partial h_2(2)} = 2$$

$$\frac{\partial h_3}{\partial z}(e^3) = e^3$$

$$\frac{\partial f}{\partial h_3} = 1$$

6 Programming

In this programming homework, we will

- implement MLP-based classifiers for the sentiment classification task of homework 1.

Skeleton Code and Structure: The code base for this homework can be found at MyClasses Files under the `hw3` directory. Your task is to fill in the missing parts in the skeleton code, following the requirements, guidance, and tips provided in this pdf and the comments in the corresponding .py files. The code base has the following structure:

- `mlp.py` reuse the sentiment classifier on movie reviews you implemented in homework 1, with additional requirements to implement MLP-based classifier architectures and forward pass .
- `main.py` provides the entry point to run your implementations `mlp.py`
- `hw3.md` provides instructions on how to setup the environment and run each part of the homework in `main.py`

TODOs — Your tasks include 1) generate plots and/or write short answers based on the results of running the code; 2) fill in the blanks in the skeleton to complete the code. We will explicitly mark these plotting, written answer, and filling-in-the-blank tasks as **TODOs** in the following descriptions, as well as a `# TODO` at the corresponding blank in the code.
TODOs (Copy from your HW1). We are reusing most of the `model.py` from homework 1 as the starting point for the `mlp.py` - you will see in the skeleton that they look very similar. Moreover, in order to make the skeleton complete, for all the `# TODO (Copy from your HW1)`, please fill in the blank below them by copying and pasting the corresponding implementations you wrote for homework 1 (i.e. the corresponding `# TODO` in homework 1.)

Submission: Your submission should contain two parts: 1) plots and short answers under the corresponding questions below; and 2) your completion of the skeleton code base, in a `.zip` file

6.1 MLP-based Sentiment Classifier

In both homework 1 & 2, our implementation of the `SentimentClassifer` is essentially a single-layer feedforward neural network that maps input features directly to 2-dimensional output logits. In this part of the programming homework, we will expand the architecture of our classifier to multi-layer perceptron (MLP).

6.1.1 Reuse Your HW1 Implementation

TODOs (Copy from your HW1): for all the `# TODO (Copy from your HW1)` in `mlp.py`, please fill in the blank below them by copying and pasting the corresponding implementations you wrote for homework 1 (i.e. the corresponding `# TODO` in the `model.py` in homework 1).

6.1.2 Build MLPs

Remember from the lecture that MLP is a multi-layer feedforward network with perceptrons as its nodes. A perceptron consists of non-linear activation of the affine (linear) transformation of inputs.

TODOs: Complete the `__init__` and `forward` function of the `SentimentClassifier` class in `mlp.py` to build MLP classifiers that supports custom specification of architecture (i.e. number and dimension of hidden layers)
Hint: check the comments in the code for specific requirements about input, output, and implementation. Also, check out the document of nn.ModuleList about how to define and implement forward pass of MLPs as a stack of layers.

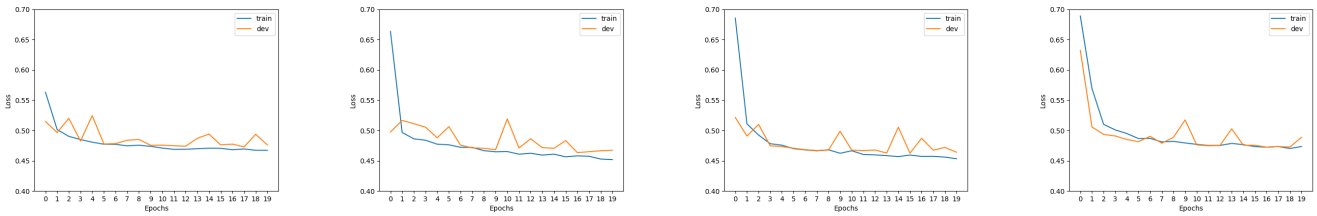
6.1.3 Train and Evaluate MLPs

We provide in `main.py` several MLP configurations and corresponding recipes for training them.

TODOs Once you finished subsubsection 6.1.2, you can run `load_data_mlp` and `explore_mlp_structures` to train and evaluate these MLPs and paste two sets of plots here:

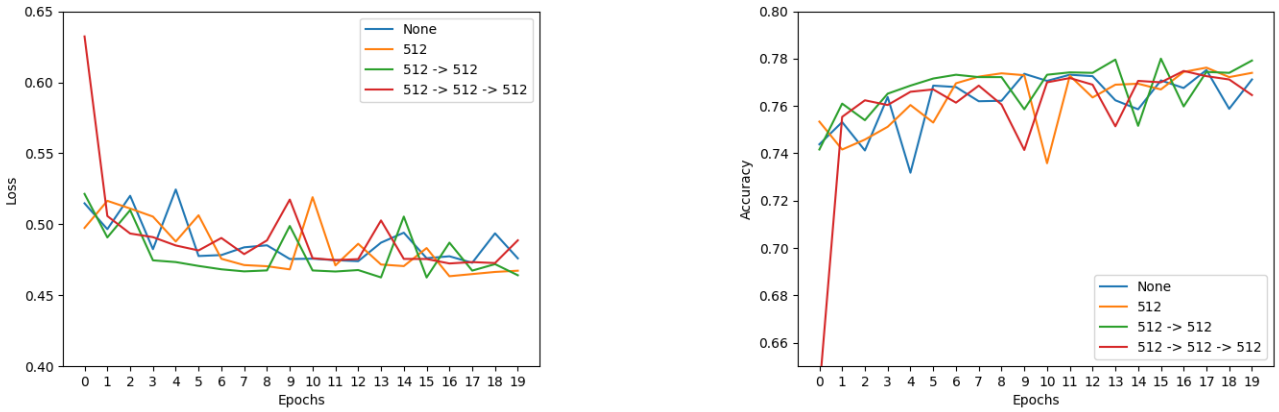
- 4 plots of train & dev loss for each MLP configuration
- 2 plots of dev losses and accuracies across MLP configurations

and describe in 2-3 sentences your findings.
Hint: what are the trends of train & dev loss and are they consistent across different configurations? Is deeper models always better? Why?
your plots and answer:
Answer: *We can see off the models below a quite surprising result that smaller models seem to perform better having generally*



(a) No Hidden Layer (b) 1 Hidden Layer (c) 2 Hidden Layers (d) 3 Hidden Layers

Figure 2: loss of different MLP architectures



(a) dev loss of different MLP architectures (b) dev acc of different MLP architectures

Figure 3: loss and acc of different MLP architectures

higher accuracy and lower loss. One potential reason for this is Vanishing and exploding gradients where increased layers result in reduced ability to train models. We can also see that the initial accuacies of the smaller models perform better potentially due to ability to train smaller models quicker given less parameters.

6.1.4 Embrace Non-linearity: The Activation Functions

Remember we have learned why adding non-linearity is useful in neural nets and gotten familiar with several non-linear activation functions both in the class and section 3. Now it is time to try them out in our MLPs!
Note: for the following TODO and the TODO in subsubsection 6.1.5, we fix the MLP structure to be with a single 512-dimension hidden layer, as specified in the code. You only need to run experiments on this architecture.

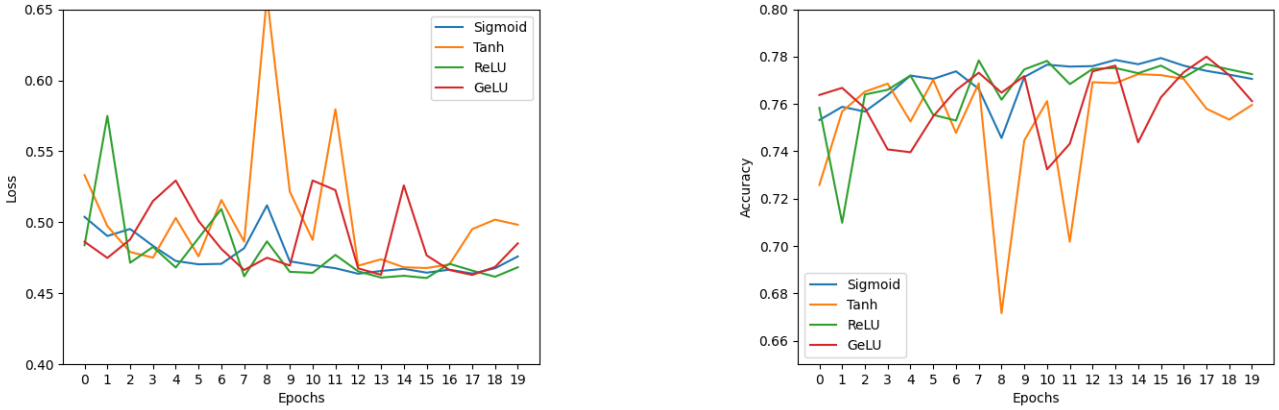
TODOs: Read and complete the missing lines of the two following functions:

- `__init__` function of the `SentimentClassifier` class: define different activation functions given the input `activation` type.
  Hint: we have provided you with a demonstration of defining the Sigmoid activation, you can search for the other `nn.<activation>` in PyTorch documentation.

- `explore_mlp_activations` in `main.py`: iterate over the activation options, define the corresponding training configurations, train and evaluate the model, and visualize the results. Note: you only need to generate the plots of dev loss and dev acc across different configurations, by calling `visualize_configs`, you **do not** need

to plot the train-dev loss curves for each configuration (i.e. no need to call `visualize_epochs`). We provide you with a few choices of common activation functions, but feel free to try out the others.
Hint: You can refer to `explore_mlp_structure` as a demonstration of how to define training configurations with fixed hyper-parameters & iterate over hyper-parameters/design choices of interests (e.g. hidden dimensions, choice of activation), and plot the evaluation results across configurations.

Once you complete the above functions, run `explore_mlp_activations` and paste the two generated plots here. Describe in 2-3 sentences your findings.
your plots and answer:



(a) dev loss of different activations

(b) dev acc of different MLP architectures

Figure 4: loss and acc of different activations

Answer: *Based on results we see Sigmoid and ReLu are the best activation functions on average with Sigmoid generally performing better. We also can see that That these functions tend to be the more stable of the four.*

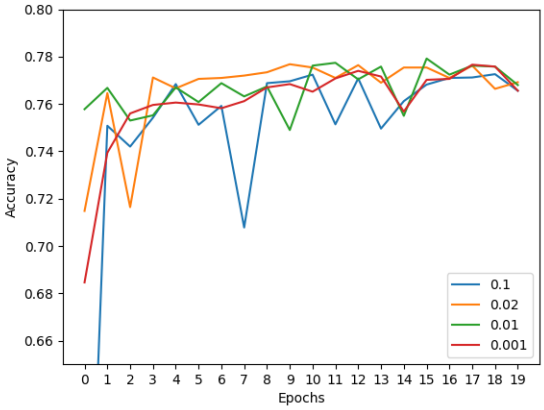
6.1.5 Hyper-parameter Tuning: Learning Rate

The training process mostly involves learning model parameters, which are automatically performed by gradient-based methods. However, certain parameters are "unlearnable" through gradient optimization while playing a crucial role in affecting model performance, for example, learning rate and batch size. We typically refer to these parameters as *Hyper-parameters*.

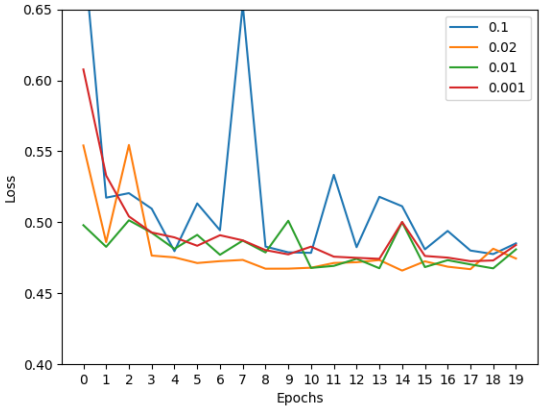
We will now take the first step to tune these hyper-parameters by exploring the choices of one of the most important one - learning rate, on our MLP. (There are lots of tutorials on how to tune the learning rate manually or automatically in practice, for example this note can serve as a starting point.)

TODOs: Read and complete the missing lines in `explore_mlp_learning_rates` in `main.py` to iterate over different learning rate values, define the training configurations, train and evaluate the model, and visualize the results. Note: same as above, you only need to generate the plots of dev loss and dev acc across different configurations, by calling `visualize_configs`, you **do not** need to plot the train-dev loss curves for each configuration (i.e. no need to call `visualize_epochs`). We provide you with the default learning rate we set to start with, and we encourage you to add more learning rate values to explore and include in your final plots curves of **at least 4 different representative learning rates.**
Hint: again, you can checkout `explore_mlp_structure` as a demonstration for how to perform hyper-parameter search.
Once you complete the above functions, run `explore_mlp_learning_rates` and paste the two generated plots here. Describe in 2-3 sentences your findings.
your plots and answer:

Answer: *In the long run given a high number of epochs there isnt much difference between learning rates which makes sense as they should all be converging to the same value. However in the short term we see high learning rates results in more spratic changes but in general faster convergence while lower learning rates are the opposite.*



(a) dev loss of different learning rates



(b) dev acc of different MLP architectures

Figure 5: loss and acc of different learning rates